

Peter Kjær Willendrup^{1,2}, Emmanuel Farhi³, Mads Bertelsen², Torben Roland Nielsen², Tobias Weber⁴, Erik Bergbäck Knudsen^{1,5}, Jakob Garde⁶, Tobias Weber⁴

¹Technical University of Denmark / Danmarks Tekniske Universitet (*Department of Physics*) ²European Spallation Source ERIC (*Data Management and Software Center*) , ³Synchrotron SOLEIL, ⁴Institut Laue Langevin, ⁵Copenhagen Atomics, ⁶Technical University of Denmark / Danmarks Tekniske Universitet (*Department of Electrical Engineering*)

Speeding up legacy:

GPU-accelerating the McStas instrument simulation code using OpenACC.



Agenda

- What is McStas in two slides
- Why and how was McStas ported to GPU's
- How well (fast) does it work?
- HOWTO steps:
Porting an instrument / component
- Conclusions

McStas Introduction

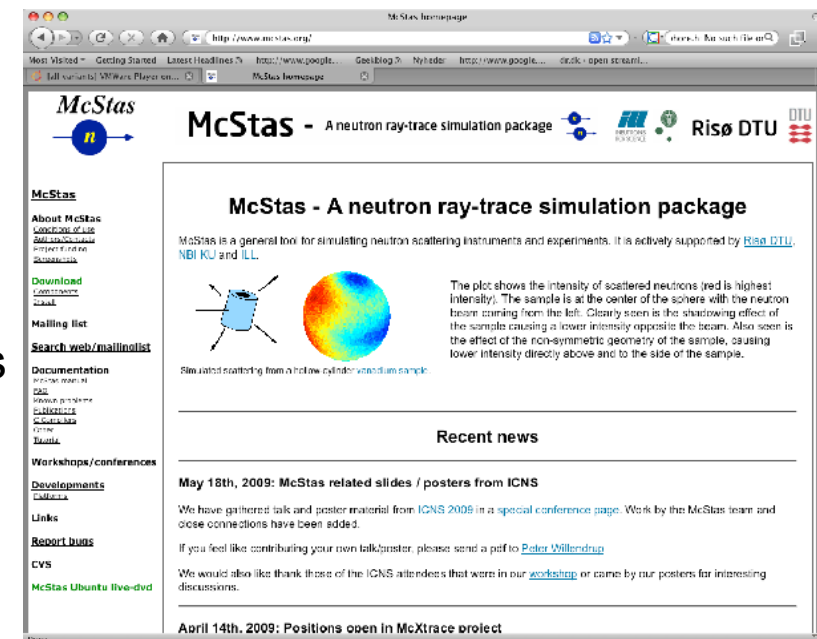


GNU GPL license
Open Source

- Flexible, general simulation utility for neutron scattering experiments.
- Original design for Monte carlo Simulation of triple axis spectrometers
- Developed at DTU Physics, ILL, PSI, Uni CPH, ESS DMSC
- V. 1.0 by K Nielsen & K Lefmann (1998) RISØ
(work initiated in 1997, 25 year project anniversary, 2023 anniversary release)
- Small, dedicated team, many contributions from users, students



Similar for X-rays, see <http://www.mcxtrace.org> - we share code base, tools and infrastructure.



Project website at

<http://www.mcstas.org>

mcstas-users@mcstas.org mailinglist

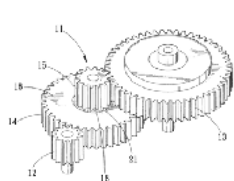
Software facts

- Portable



- CPU/MPI/GPU
- ~ 250 comps
- ~ 250 instrs

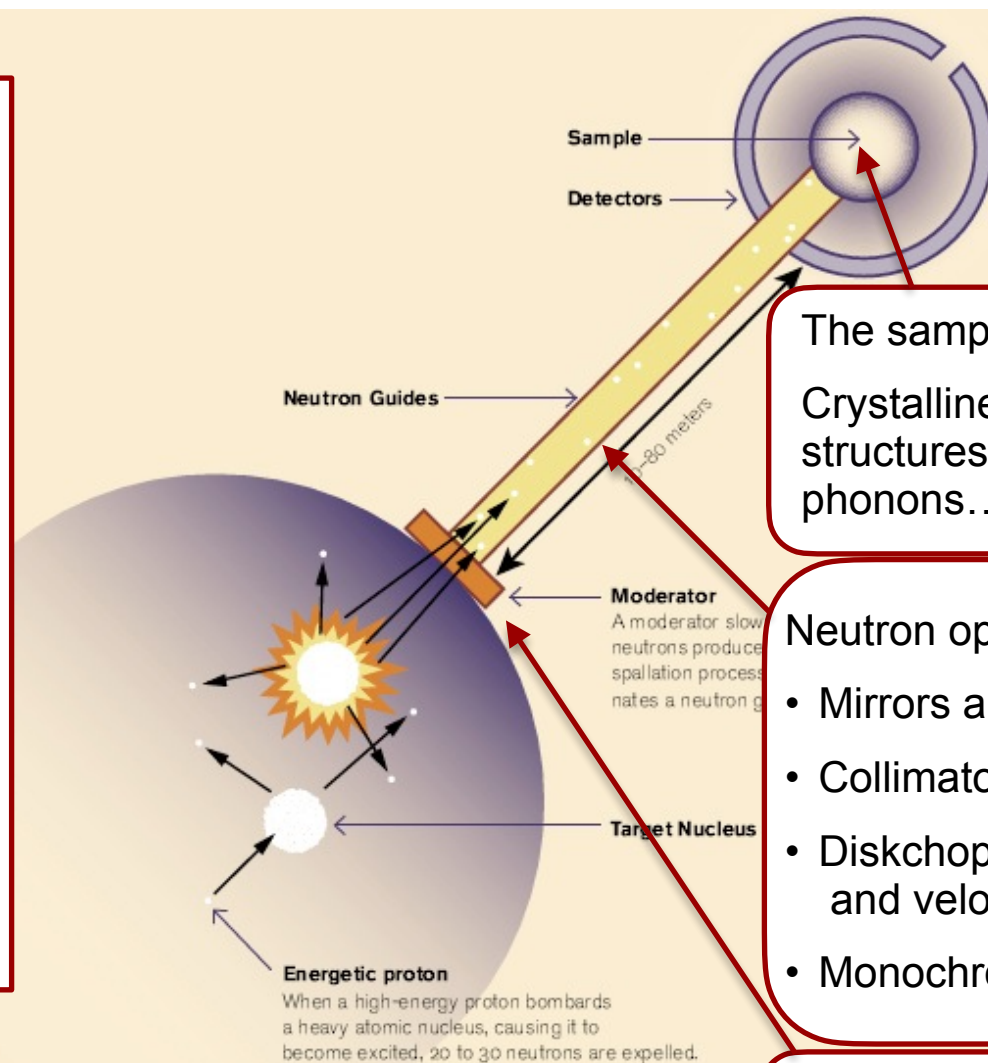
- D. S. L. domain specific language



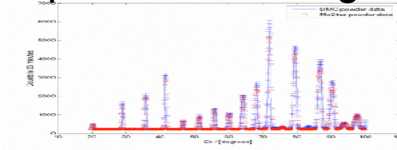
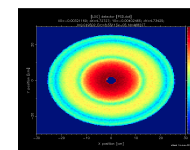
Code generation

- C-code
- Binary prog

Components of neutron instruments

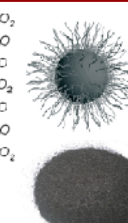
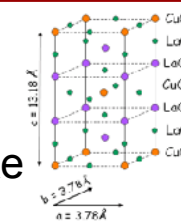


Detectors are “monitors” in McStas. Mostly they act as “perfect probes” and can be positioned thought gathering 1D/2D/ event lists...



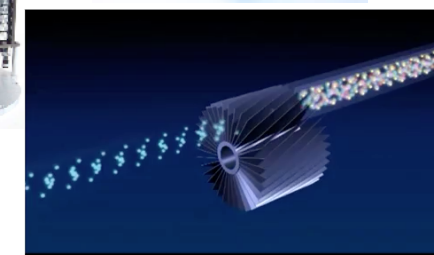
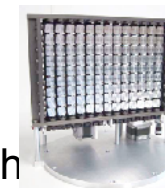
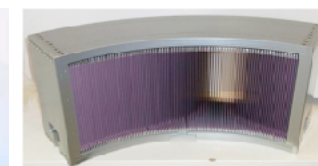
The sample:

Crystalline, powders, liquids, micelles, structures to image, inelastic features like phonons...



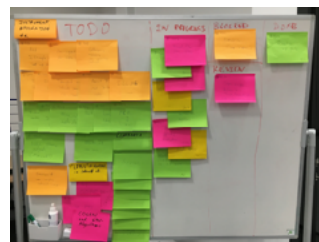
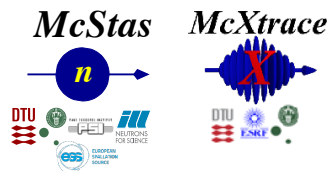
Neutron optics include things like:

- Mirrors and guides
- Collimators and slits
- Diskchoppers, Fermi ch and velocity selectors
- Monochromators/Analysers



In McStas the moderator is the “source”

Main events on timeline of road toward GPU



2017: E. Farhi
initial cogen
modernisation

Fall 2018 onwards:
J. Garde further cogen
modernisation and
restructuring

October 2019 onwards:
J. Garde & P. Willendrup:
New RNG, test system, multiple
functional instruments.

January 2020:
One-week local
hackathon @ DTU
with McCode & RAMP teams

November 2020
Virtual Hackathon,
setting release scope



March 2018: Participation at
Dresden Hackathon. 1st "null"
instrument prototype runs.

October 2019:
Participation at **Espoo Hackathon**.
First meaningful data extracted.
Work on cogen and realising
we need another RNG.

November-
December 2019:
First good look at
benchmarks and
overview of what
needs doing for first
release with limited
GPU support.

2020 1st **Corona** lockdown
P. Willendrup & E. Knudsen
continue work on comp and
cogen

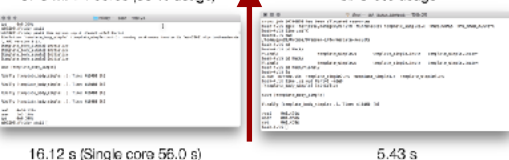
December 15th 2020
McStas 3.0 release!

November 24th, 2021
McStas 3.1 release!

McStas / McXtrace instrument simulation

CPU MPI 4 cores (95 % usage)

GPU 5% usage



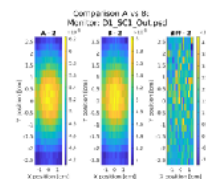
mentor: Vishal Metha 

hackathon org.:
Guido Juckeland

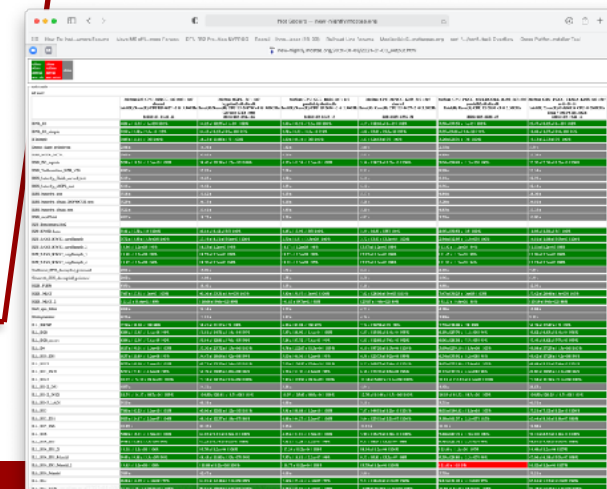


mentor: Christian Hundt 

hackathon org.:
Sebastian Von Alfthan



February 2020:
First release
McStas 3.0beta
with GPU
support was
released
to the public



Instrument	Version	Author	Release Date	Release Type	Release Notes
McStas	3.0	P. Willendrup	2020-12-15	Stable	First release with GPU support
McStas	3.0beta	P. Willendrup	2020-02-01	Beta	First release with GPU support
McStas	3.1	P. Willendrup	2021-11-24	Stable	Next release

McStas 3.2
release
is expected in
the fall of 2022

GPU-computing “101”

- LOTS of slowish processor-cores
(aka. massively parallel)
- BUT: Limited bandwidth in CPU / GPU exchange
(PCIe...)
- Main HW providers today are  NVIDIA,  , 
- Software-frameworks
 - CUDA (NVIDIA-specific)
 - Accessible in C, ++, Fortran, shared library
 - OpenCL (hw-agnostic)
 - C-like programming language with own syntax
 - **OpenACC (almost NVIDIA-specific, but extending**
 - **#pragma pre-compiler mechanism, accessible in C, ++, Fortran**
 - **“High-level, compiler-driven CUDA”**
 - OpenMP is picking up GPU-support...
 - Intel oneAPI
 - Claims to be a unified approach to CPU/GPU/MPI/... I didn't get around to really try it yet... ;-)

Why consider GPU's for McStas in the first place?

- GPU's have great potential for speedups within “intrinsically parallel” problems
- McStas is already “embarrassingly parallel”: every neutron is independent
- TCO or “FLOP / energy” can be **greener** / cheaper than for CPU's

Why OpenACC?

- Was retrofitted elegantly to our “old” sw framework”
- **Identical code** runs now on both **CPU** and **GPU**

Foreseen use-case?

- Pre- or in-experiment help tool needs to be “as fast as the experiment”. (Modern day spallation sources with event-mode and sample sim. are a challenge.)



GPU support in McStas, so far is effectively NVIDIA-specific!

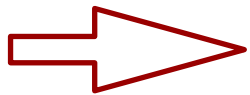


Anatomy of a McStas GPU run (*)

- Init, geometry, files etc. read on CPU
 - MPI if needed
- Memory-structures
 - Built on CPU
- Marked for transfer to GPU (`#pragma acc declare create etc.`)
- Initialised and synced across
- Trace-loop is a `#pragma acc parallel loop`
 - Calculation performed entirely on GPU
 - Component structs (incl. e.g. monitor-arrays) synced across
- Finally and Save runs on CPU
 - MPI merge if needed

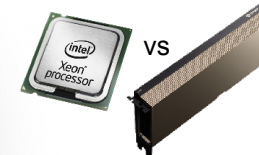


No printf's etc. available
on GPU, automatically
suppressed by `#defines`



The GPU user experience is almost identical to that of running on CPU!

Maximum performance indication on NVIDIA A100 (Ampere)

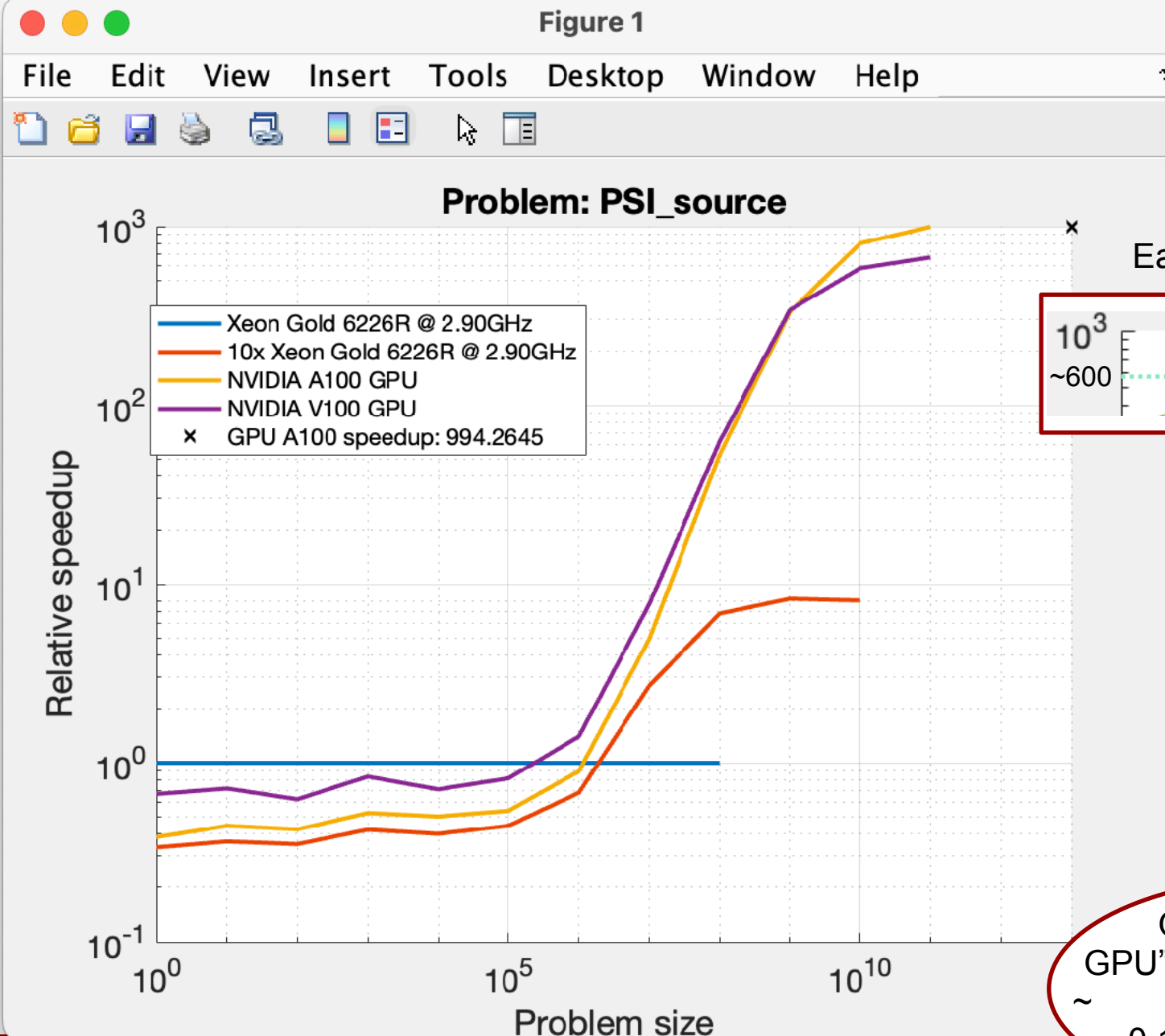


Maximal speedup: ~1000

Earlier dataset from V100 ~600

Idealised instrument with source and monitor only - i.e. without any use of the ABSORB macro.

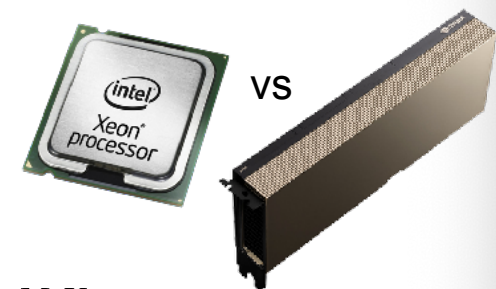
(Good indication of maximal speedup achievable.)



Execution speedups renormalised to wall-clock of single-core gcc standard simulation,

A100 run is ~ 1000 times faster than a single-core CPU run

Older "Gamer-GPU" e.g. GeForce 1030 is ~ 0.1 V100 or 0.05 A100



Real-world problem 1: BNL_H8 TAS models with / without SPLIT:

factor of **28-172** speedup
(SPLITs makes the instrument less 'intrinsically' parallel)

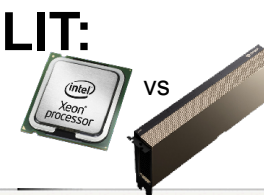


Figure 11

File Edit View Insert Tools Desktop Window Help



Problem: BNL_H8

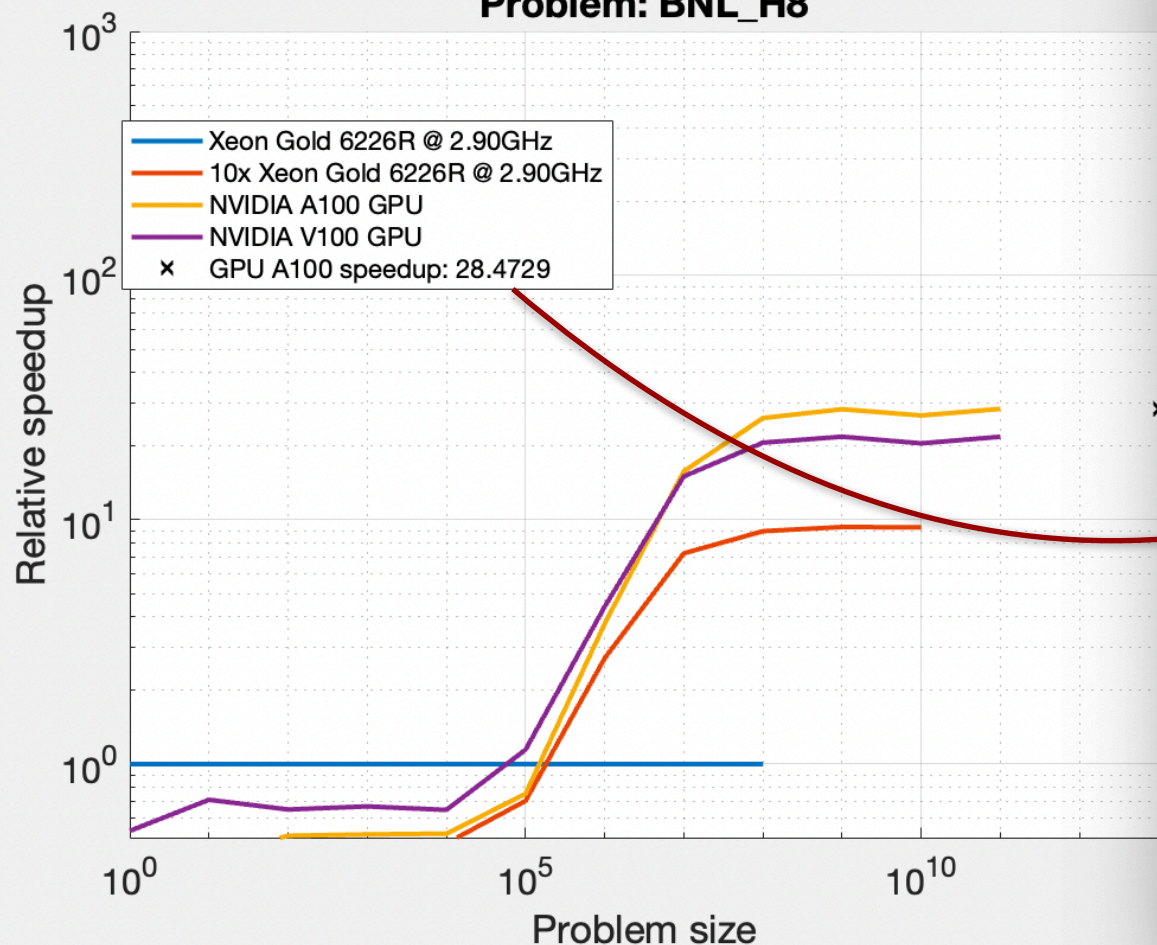
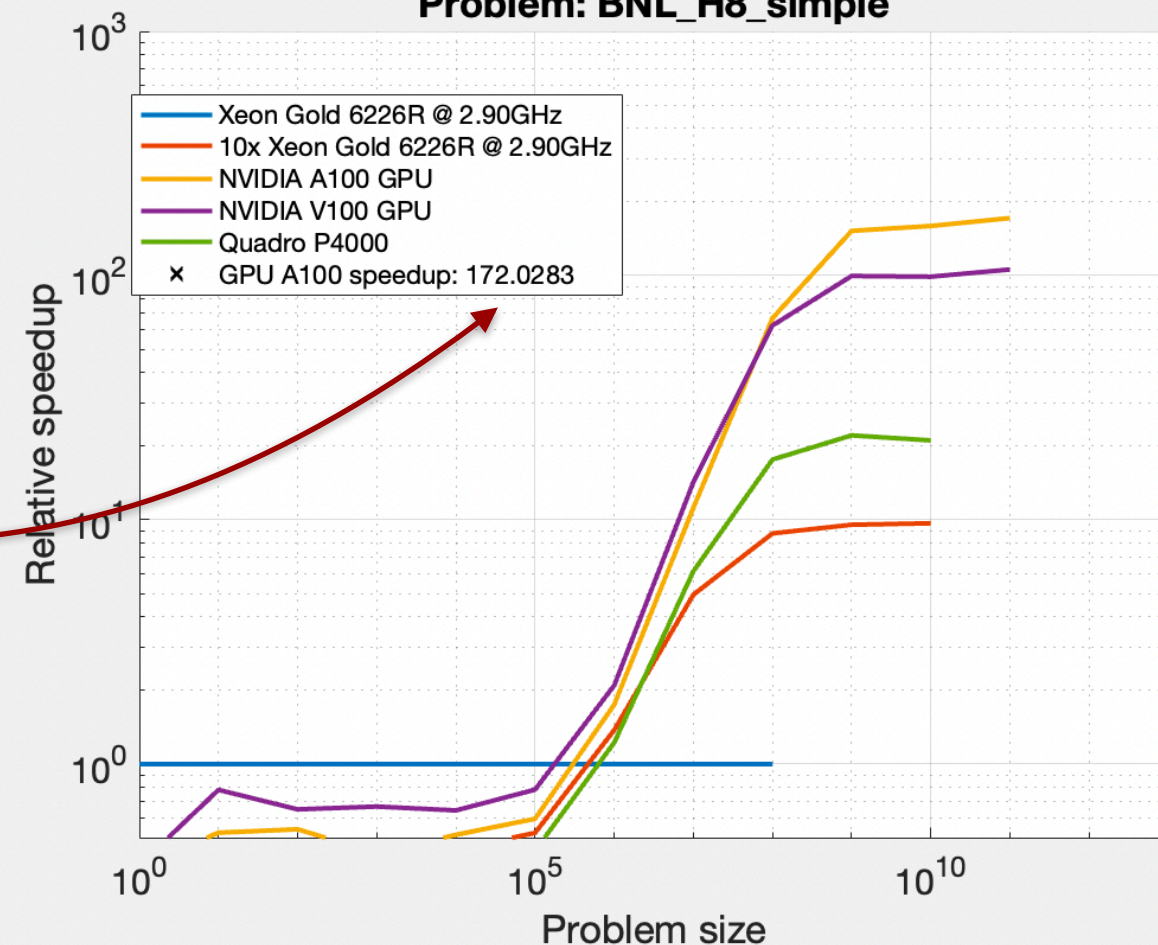


Figure 12

File Edit View Insert Tools Desktop Window Help



Problem: BNL_H8_simple



Real-world problem 2: PSI_DMC diff models with / without SPLIT: factor of 21-74 speedup

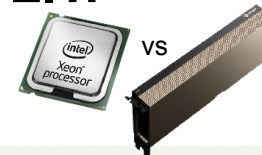


Figure 2

File Edit View Insert Tools Desktop Window Help



Problem: PSI_DMC

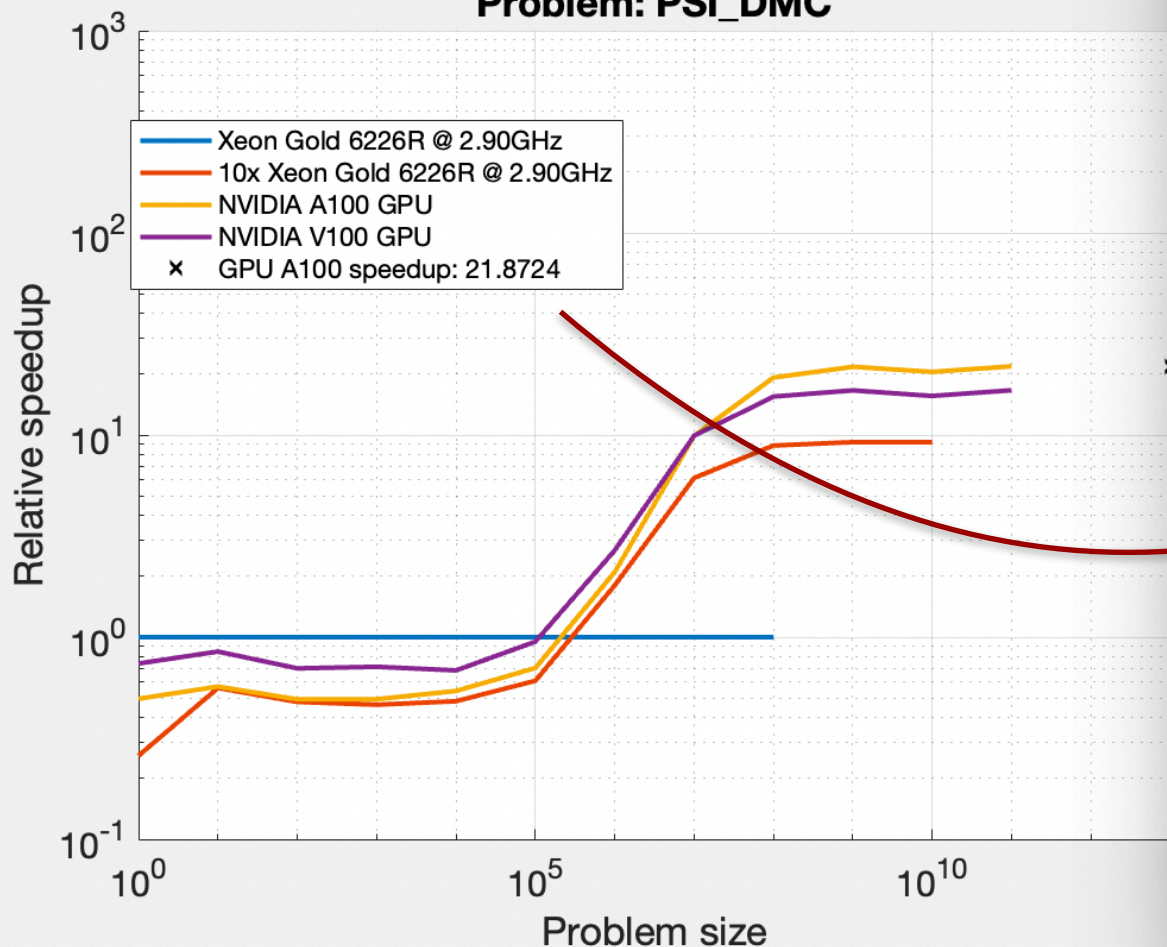
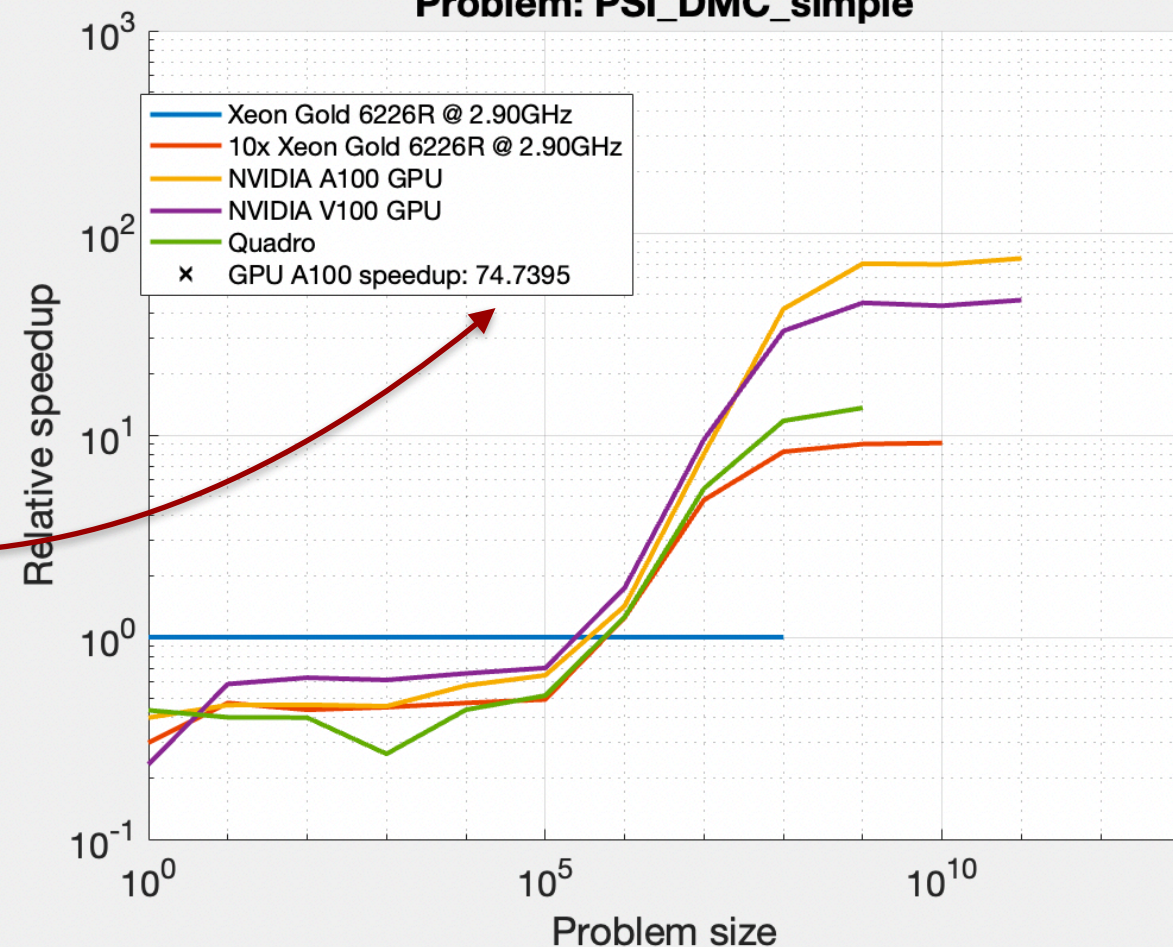


Figure 3

File Edit View Insert Tools Desktop Window Help

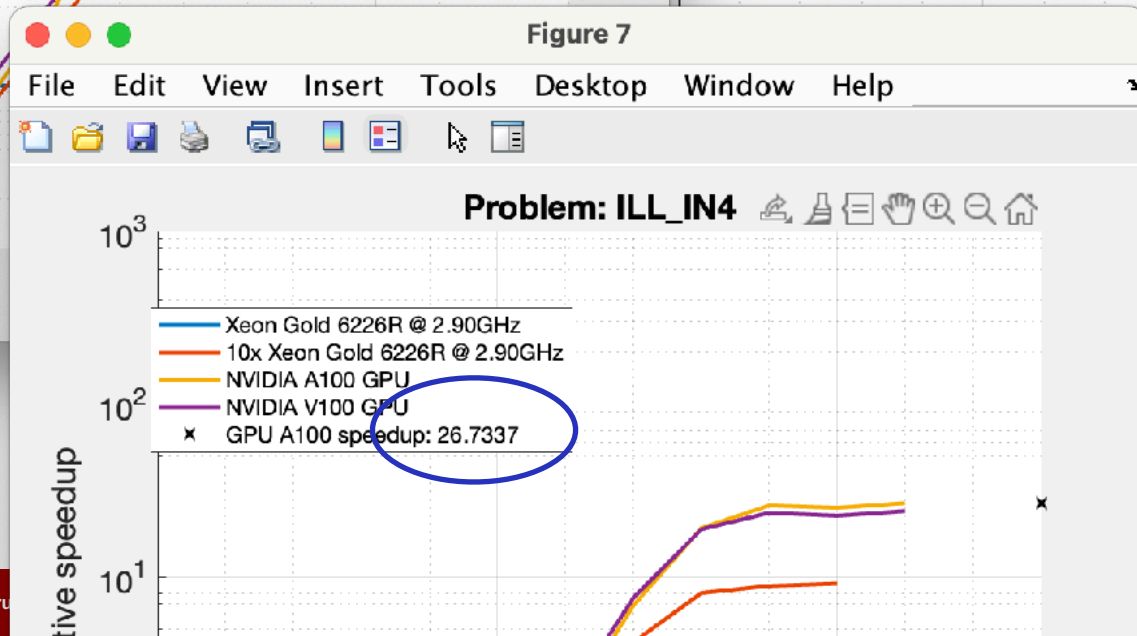
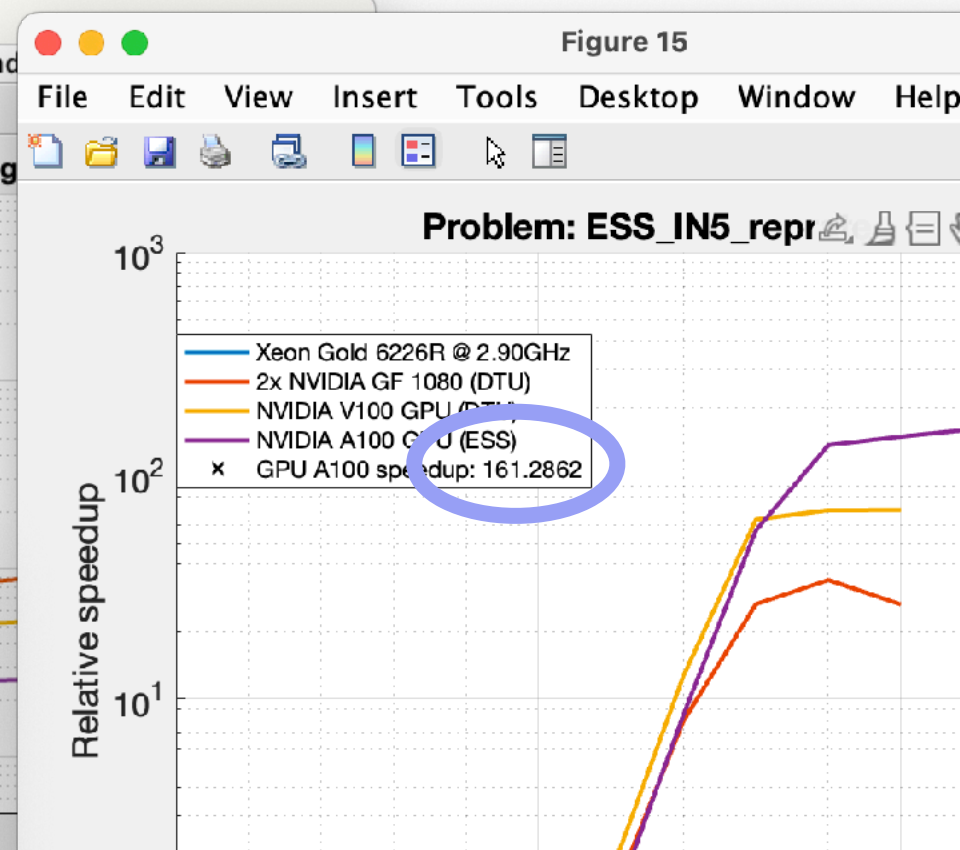
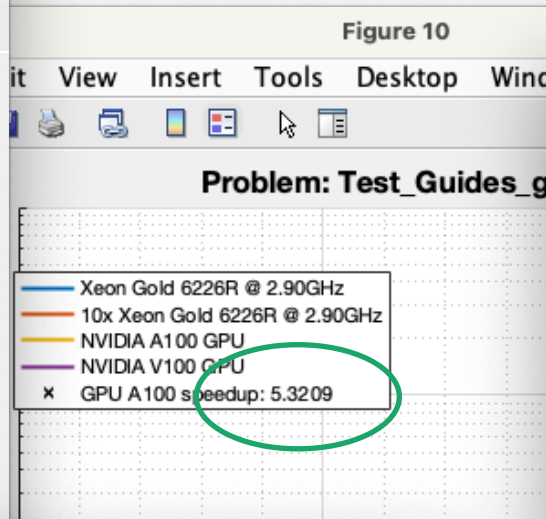
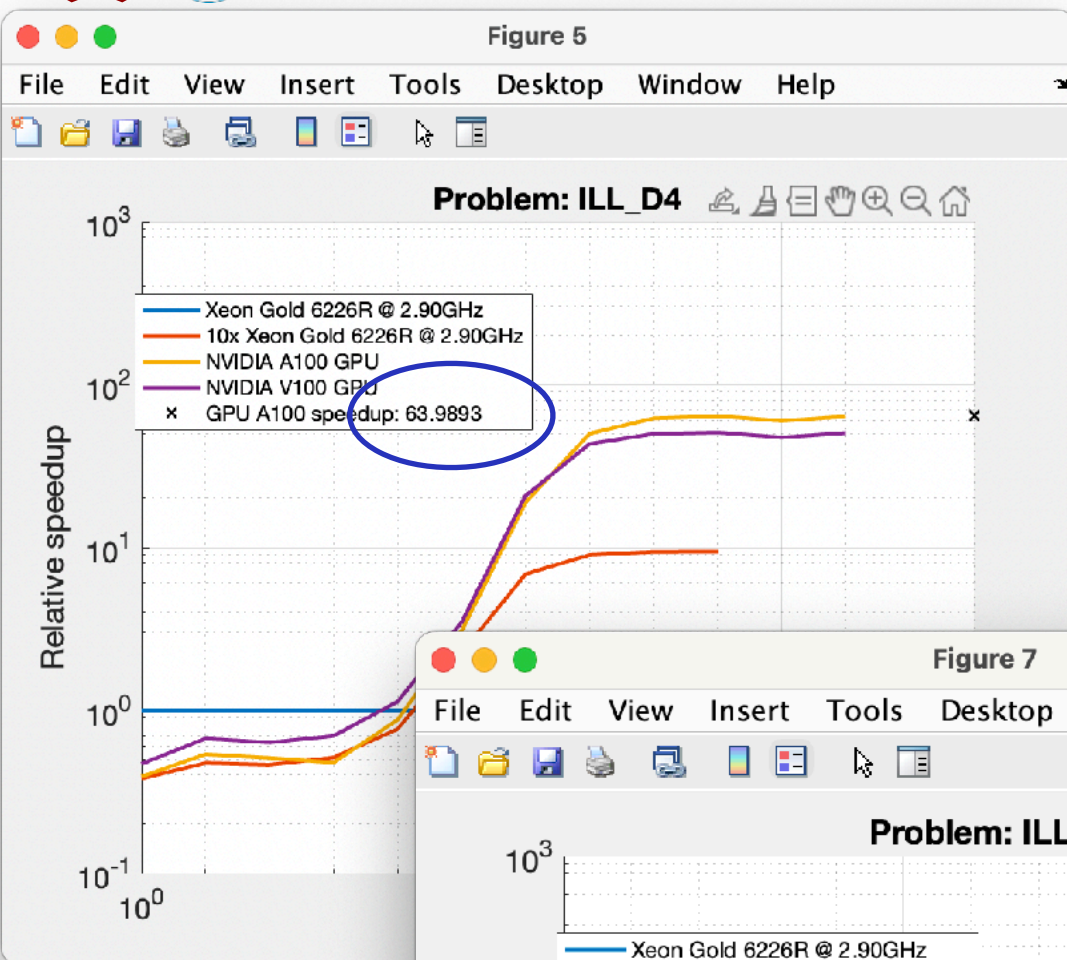


Problem: PSI_DMC_simple



Real-world problems 3:

“sample-only” sims often ~ factor of 50
 “full instruments” ~ factor of 20-30-60, but **160 seen**
 “optics-only” surprisingly ~ factor of 5 ??

**Main message:**

Here is room for more, optimisation ongoing!!!

- * handle splits better
- * investigate “key” components
- * structural code-changes (comp USERVARS etc)

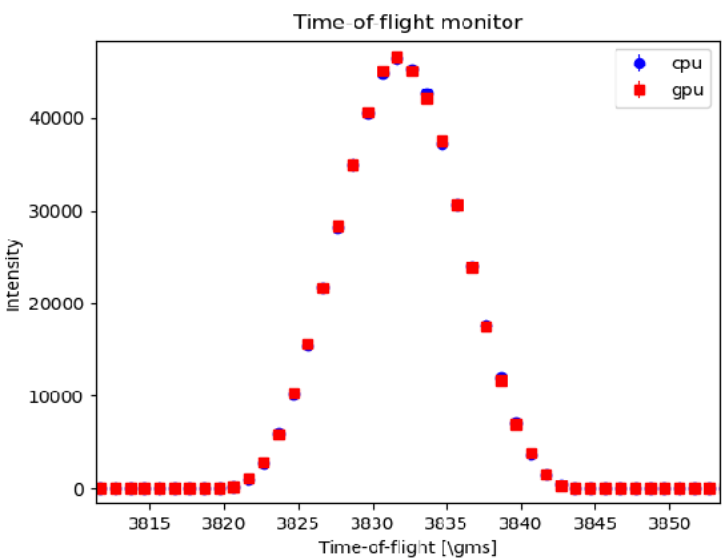


Team: Garrett E. Granroth, Fahima Islam, Thomas Huegle, Jiao Lin, Peter Willendrup

- Overall project goal: Deliver realistic instrument simulations to users on the same time scale as a neutron scattering measurement.
- Currently the most time is spent simulating up to the sample (The Incident beam)
- Beam line simulations for SNAP, GPSANS, and ARCS have been updated for McStas 3.X. (They can now run on NVIDIA gpus as well as cpus.)
- The GPSANS update was straight forward as it leveraged updated McStas components
- The SNAP, SEQUOIA and ARCS simulations have custom components so they were more effort.
- The openacc implementation in McStas 3.X streamlined the development process.
- Simulations run more than 100x faster on an NVIDIA A100 than a cpu.
- With all 8 gpus on an NVIDIA A100, simulations can run much faster than the duration of a single neutron scattering measurement

Details of Simulation Speed increase

Instrument	Time on 1 CPU (s)	Time on 1 GPU (s)	Speed up (x)
ARCS	103920	58	1791
GPSANS	3380	16	211
SNAP	20592	60	343



Beam monitor from ARCS simulation shows no statistically significant difference between cpu and gpu simulations

Porting an instrument - HOWTO

see also <https://github.com/McStasMcXtrace/McCode/wiki/HOWTO%3A-Modifying-a-McStas-2-instrument-for-use-under-McStas-3>

```
* %Example: lambda=5 order=1 directbeam=0 SPLITS=10 Detector: Sph_mon_I=7.2e+7
*
* %Parameters
* lambda: [Angs] Central wavelength emitted from source
* dlambda: [Angs] Width of wavelength spectrom emitted from source
* L1: [m] Source-sample distance
* directbeam: [1] Suppress direct beam or not
* reflections: [str] File name for reflection list
* SPLITS: [1] Number of SPLIT's before sample
*
* %End
*****
DEFINE INSTRUMENT Test_SX(lambda=5, dlambda=9.8, L1=30, int directbeam=0, beam

DECLARE %{
  int DirectBeam;
  int Type;
%}

/* The INITIALIZE section is executed when the simulation starts
/* (C code). You may use them as component parameter values.
INITIALIZE
%{
  DirectBeam = directbeam;
%}

/* Here comes the TRACE section, where the actual
/* instrument is defined as a sequence of components.
TRACE

REMOVABLE COMPONENT Origin = Progress_bar()
  AT (0,0,0) ABSOLUTE
EXTEND %{
  Type=0;
%}

/* source with constant flux */
REMOVABLE COMPONENT Source = Source_gen(
  radius = 0.11, dist = L1, focus_xw = 0.01, focus_yh = 0.01,
```

```
* %Example: lambda=5 order=1 directbeam=0 SPLITS=10 Detector: Sph_mon_I=7.2e+7
*
* %Parameters
* lambda: [Angs] Central wavelength emitted from source
* dlambda: [Angs] Width of wavelength spectrom emitted from source
* L1: [m] Source-sample distance
* directbeam: [1] Suppress direct beam or not
* reflections: [str] File name for reflection list
* SPLITS: [1] Number of SPLIT's before sample
*
* %End
*****
DEFINE INSTRUMENT Test_SX(lambda=5, dlambda=9.8, L1=30, int directbeam=0, beam

DECLARE %{
  int DirectBeam;
  #pragma acc declare create(DirectBeam)
%}

USERVARS %{
  int Type;
%}

/* The INITIALIZE section is executed when the simulation starts
/* (C code). You may use them as component parameter values.
INITIALIZE
%{
  DirectBeam = directbeam;
  #pragma acc update device(DirectBeam)
%}

/* Here comes the TRACE section, where the actual
/* instrument is defined as a sequence of components.
TRACE

REMOVABLE COMPONENT Origin = Progress_bar()
  AT (0,0,0) ABSOLUTE

/* source with constant flux */
```

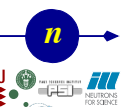
McXtrace



* Particle “flags” must go in **USERVARS** (automatically initialised to 0 in 3.x)

* DECLARE-vars used within instrument TRACE need **#pragma**’s

McStas



* Monitor_nD user-variables-identifiers are **string** type

user1=flag → user1=“flag”

If you can’t make it work, please write mcstas-users@mcstas.org or define a [GitHub issue](#)

Porting a component - HOWTO

see also <https://github.com/McStasMcXtrace/McCode/wiki/HOWTO%3A-Modifying-a-McStas-2-component-for-use-under-McStas-3>

1. **DEFINITION PARAMETERS is not supported.** Vars must become SETTING PARAMETERS, specifically:
 - Simply move string vars
 - Lists/array-pointers need the vector type e.g. vector a={1,2,3,4} or b=c where c is an instrument-level array/pointers. (Base type in vector is a double.)
2. **DECLARE must have “simple” content**, i.e. vars without initialisation each alone on a line:

```
DECLARE %{  
    double a,b;  
    %}
```



```
DECLARE %{  
    double a;  
    double b;  
    %}
```

- and initialise them in INITIALIZE

3. **SHARE-based TRACE-functions** that pick random numbers **MUST include the “particle”** in the footprint:
 - double my_function(double a, int b, double*c, **_class_particle*** _particle);
 - this forwards the RNG state (carried with each particle)
4. **DECLARE-parameters should not be used to store particle-derived information...**
 - Use a local TRACE-scope var instead
 - Ensure this by checking that CPU and GPU runs are identical if run with the same seed.
5. If you are using **external libs**, e.g. GSL or function pointers, your code can not run on GPU.
 - You may put the **NOACC keyword** in the component header, this forces execution on CPU only.

If you can't make it work, please write mcstas-users@mcstas.org or define a [GitHub issue](#)

All instruments (>250) distributed with McStas 3.x can utilise NVIDIA GPUs.

- please use as inspiration!!

1. You need an NVIDIA card in your machine
2. Use Linux ;-) (or WSL 2 on windows, including relevant driver and kernel...)
3. Install the NVIDIA hpc sdk <https://developer.nvidia.com/nvidia-hpc-sdk-downloads>
4. Your McStas 3.x is preconfigured with reasonable defaults if the nvc compiler is on the PATH, i.e.

A. Single-core CPU compilation by
`mcrun -c Instrument.instr`

B. Enable MPI by
`mcrun -c --mpi=8 Instrument.instr`

C. Enable GPU by
`mcrun -c --openacc Instrument.instr`

... Or use similar settings from your mcgui.

“Do almost as usual”.

D. Combined MPI and GPU run can be achieved via
`mcrun -c --mpi=8 --openacc Instrument.instr`

Output data should look “as usual”, an instrument compiled for GPU can (currently) not output mcdisplay graphics.

If you can't make it work, please write mcstas-users@mcstas.org or define a [GitHub issue](#)

Conclusions

- It really does work nicely!
- **Code changes** much **less invasive** than envisioned!
- Use is transparent, fully integrated in mcgui / mcrun utils (on Linux or through WSL)
- It often gives a speedup of **1-2 orders** of magnitude over 1 cpu core
- Used for pre / in experiment simulations at ORNL, runs at least “real-time” wrt. experiments
- Most things work already
(we have workarounds or solutions in the pipe for the rest)
- McStas 3.1 is as of yet “fully ported” to GPU but **not fully “optimised” performance-wise**, we will try to go to another Hackathon
- Basic **compilation** with GCC 10 offloading support achieved May 2021
 - but produces 0 on detectors... ;-)
 - hope for better GCC support and non-NVIDIA cards in 1-2 years



The team, Nvidia mentors and Hackathon hosts :-)



Vishal Metha



Christian Hundt



Alexey Romanenko



Guido Juckeland



Sebastian von Alfthan

Datacenters:

- HZDR Dresden
- CSC Espoo
- DTU Lyngby
- DTU RISØ
- ESS DMSC



— Other McStas contributions —

22/08 - MONDAY			
12:30 - 18:30  Poster Section 1			
<i>Pôster</i> ↗ Poster1 - Program			
13-13 12:30 - 18:30	Flautim and Araponga - The future of neutron diffraction in Brazil 	WATCH THIS PRESENTATION PUT ON MY AGENDA	
13 - Neutron Facilities, Instrumentation and Software / Pôster <i>ALEXANDRE PINHO DOS SANTOS SOUZA; Luiz Paulo de Oliveira; Frederico Antonio Genezini; ADIMIR DOS SANTOS</i>			
13-55 12:30 - 18:30	Optimization of Wide Angle Neutron Diffractometer for Maximum Intensity 	WATCH THIS PRESENTATION PUT ON MY AGENDA	
13 - Neutron Facilities, Instrumentation and Software / Pôster <i>Fahima Fahmida Islam</i>			
13-56 12:30 - 18:30	McStasScript, a Python API for McStas 	WATCH THIS PRESENTATION PUT ON MY AGENDA	
13 - Neutron Facilities, Instrumentation and Software / Pôster <i>Mads Bertelsen</i>			
13-57 12:30 - 18:30	ANDES, a new neutron strain scanner for LAHN 	WATCH THIS PRESENTATION PUT ON MY AGENDA	
13 - Neutron Facilities, Instrumentation and Software / Pôster <i>Miguel Angel Vicente Alvarez; Martin Gonzalez Fuster; Javier Santisteban; Agustín Beceyro; Santiago Gomez; Karina Pierpauli</i>			
24/08 - WEDNESDAY			
14:30 - 16:00  Reflection and supermirrors			
Chair: Fredrik Eriksson <i>Oral</i> ↗ Room B - Program			
13-156 15:50 - 16:10	Detailed study of the neutron scattering from highly oriented pyrolytic graphite 	PUT ON MY AGENDA	
13 - Neutron Facilities, Instrumentation and Software / Oral <i>Kristine Marie Lofgren Kriggaard; Jacob Larsen; Rasmus Laurberg Hansen; Xiaoyu Wang; Jakob Lass; Jonas Okkels Birk; Marton Marko; Matthias Frontzek; Christof Niedermayer; Rasmus Toft-Petersen; Niels Bech Christensen; Kim Lefmann</i>			

— Backup slides follow —

McStas 2.x -> McStas 3.x main differences

- **Rewritten** / streamlined simplified **code-generator** with
 - Much **less generated code**
 - **improved compile time and compiler optimizations**, esp. for large instrs
 - **Much less invasive use of #define**
 - **Component sections -> functions** rather than #define / #undef
 - Much **less global variables**, instrument, component and neutron reworked to be **structures**

Advantage
of 3.0 also on
CPU



- Use of **#pragma acc ...** in lots of places (**put in place by cogen** where possible)

OpenACC

- **New random number generator** implemented
 - We couldn't easily port our legacy Mersenne Twister
 - Experimenting with curand showed huge overhead for our relative small number of random numbers
(we have hundreds or thousands of random numbers, not billions)
- Complete change to **dynamic** monitor-arrays

Pragmas in play...

- Data need to be transferred to the GPU, we use
 - `#pragma acc declare create( VAR ) // put at VAR declaration`
`double VAR;`
 - `#pragma acc update device(VAR) // after assignment`
- Main particle loop has a
 - `#pragma acc parallel loop`
`for (unsigned long pidx=0 ; pidx < innerloop ; pidx++) {`
- Any function to be evaluated on GPU needs. Put in place by code-generator whenever we can...
 - `#pragma acc routine // on fct. prototype or on actual function def.`
`_class_Source_div *class_Source_div_trace(_class_Source_div *_comp , _class_particle *_particle) {`
- If writing to VAR is necessary, this can be done *atomically* using e.g.
 - `#pragma acc atomic`
`VAR = VAR + 1; // ++ operators etc. is too complex`
- We pull data back using this mechanism
 - `#pragma acc update host(VAR)`

We also use

- * `openacc.h` e.g. for “attaching device pointers”
- * `acclmath.h` for a `math.h` GPU replacement

+ some self-made replacements for e.g. string handling that are (otherwise) not available for GPU.

Things that couldn't be done



- **Function pointers / abstract functions are not available on GPU**
 - Solutions:
 - Code around if possible (e.g. integration routine pr. specific function to be integrated...)
 - Use mechanism to do this calculation CPU-side before/after/at cost of transfers
- **Variadic functions are not available on GPU**
 - Special case: printf() and friends
- **Anonymous structs as comp pars are not available on GPU**
 - Declare struct explicitly
- **External libs generally can not be used on GPU**
 (“#pragma....” hard to add on 3rd party codes)
 - Handle in INIT / FINALLY (MCPL)
 - “NOACC” (GSL etc.)